

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY



ASSIGNMENT REPORT

COURSE: DISTRIBUTED APPLICATIONS

ENTERPRISE BUSINESS MANAGEMENT SYSTEM LOGISTICS ABC CORPORATION

Student:

Nguyen The Hien - 22127107
Le Quang Tan - 22127378
Bui Le Khoi - 22127205
Nguyen Minh Hieu - 21127742

Instructors:

Nguyen Truong Son
Pham Minh Tu

Contents

1	Use Case Specifications	2
2	Database Schema Design	8
2.1	Architectural Overview and Notation Conventions	8
2.1.1	Storage Architecture	8
2.1.2	Relationship Notation Conventions	9
2.2	Detailed Specification for Each Service Group	9
2.2.1	Contract Service (Customer & Contract Management Service)	9
2.2.2	Operation Service (Operations & Actual Volume Service)	10
2.2.3	Pricing Service (Price List & Service Catalog Management Service)	11
2.2.4	Billing Service (Billing Sheet Service)	12
2.2.5	Workflow Service (Approval Workflow Service)	13
2.2.6	Supporting Service (Support & Shared Utilities Service)	14
3	Database Sequence Specifications	16
3.1	UC-01 Enterprise Customer Management	16
3.2	UC-02 Contract Lifecycle Management	17
3.3	UC-04 Service Price List Management	18
3.4	UC-05 & UC-06 Billing Generation	19
3.5	UC-08 Asynchronous E-Signing	20
3.6	UC-09 & UC-10 Notification and Audit Logging	21
4	UI/UX	22

1 Use Case Specifications

UC-01: Enterprise Customer Management

Use Case	Enterprise Customer Management (Req 4.1)
Actor	Sales Executive, System Administrator, Customer
Precondition	1. Sales Executive is authenticated and authorized in the system. 2. Customer legal tax code, company details, representative, and contact information are available.
Postcondition	1. Customer profile is created or updated with status (ACTIVE or SUSPENDED). 2. Customer information is available for contract, price list, and billing lookup through Customer ID.
Main path (M)	1. Sales Executive enters customer details (Tax code, Name, Address, Representative, Contact information). 2. System validates the tax code format and verifies its uniqueness. 3. System stores the customer profile in the customers table. 4. System publishes a CUSTOMER_CREATED event to Kafka Message Broker. 5. System displays a success message and enables related contract, price list, and billing lookup.
Alternative path	(A1) Duplicate Tax Code Detected: 2a. System detects that the tax code already exists. 3a. System rejects the request and returns HTTP 409 Conflict. (A2) Customer Suspension: 1a. Administrator updates customer status to SUSPENDED. 2a. System prevents creation of new contracts, price lists, or billing records for that customer.

UC-02: Contract Lifecycle Management

Use Case	Contract Lifecycle Management (Req 4.2)
Actor	Sales Executive, Legal Staff, Board of Directors
Precondition	1. Target customer exists with ACTIVE status. 2. Contract information (effective dates, value, payment terms, service terms, attachments) is available.
Postcondition	1. Contract is created with its current status stored in the <code>contracts</code> table. 2. Approval workflow information is maintained by the Workflow Service until the contract reaches the ACTIVE status.
Main path (M)	1. Sales Executive drafts a contract with customer information, validity period, payment terms, service terms, and PDF attachment. 2. Sales Executive submits the contract to the Workflow Service. 3. Workflow Service assigns the current approval task to the appropriate reviewer and records the approval history. 4. Workflow Service updates the contract status according to the approval result; when the effective date is reached, the contract status becomes ACTIVE. 5. Sales Executive may initiate contract extension or cancellation when required.
Alternative path	(A1) Direct Amendment Blocked on Active Contract: 1a. Sales Executive attempts to modify an ACTIVE contract. 2a. System rejects the modification and requires creation of a Contract Appendix. (A2) Contract Rejected During Approval: 3a. Reviewer rejects the contract. 4a. Workflow Service updates the workflow status to <code>REVISION_REQUESTED</code> and notifies the author.

UC-03: Contract Appendix Lifecycle & Revision Management

Use Case	Contract Appendix Lifecycle & Revision Management (Req 4.3)
Actor	Sales Executive, Legal Staff, Board of Directors
Precondition	1. Parent Contract exists with ACTIVE status. 2. Appendix information including updated values or supplementary services is prepared.
Postcondition	1. Contract Appendix is created and linked to the parent contract. 2. Billing generated after the appendix effective date uses the updated appendix information.
Main path (M)	1. Sales Executive selects an active contract and creates a Contract Appendix. 2. Sales Executive specifies appendix information including <code>before_value</code>, <code>after_value</code>, validity changes, or additional services. 3. Sales Executive submits the appendix to Workflow Service. 4. After approval, the appendix status becomes EFFECTIVE on its effective date. 5. System applies the updated appendix information to subsequent billing transactions.
Alternative path	(A1) Invalid Parent Contract State: 1a. Parent contract is expired, cancelled, or not active. 2a. System rejects appendix creation. (A2) Appendix Rejected: 4a. Workflow Service marks the appendix as rejected. 5a. Parent contract information remains unchanged.

UC-04: Service Price List Management & Overlap Validation

Use Case	Service Price List Management & Overlap Validation (Req 4.4)
Actor	Sales Executive, Board of Directors, Contract Service
Precondition	1. Target Customer, Contract, or Service scope is selected. 2. Effective date range is specified.
Postcondition	1. Price List is published with a new version. 2. System validates that no effective date overlap exists before publishing the new version.
Main path (M)	1. Sales Executive configures price list items. 2. Sales Executive specifies the effective date range. 3. System validates overlapping date ranges against existing EFFECTIVE price lists. 4. After approval, the new price list becomes EFFECTIVE; previous versions are marked as SUPERSEDED. 5. System stores all versions for historical reference.
Alternative path	(A1) Effective Date Overlap Detected: 3a. System detects an overlapping effective period. 4a. System rejects the new price list and displays the conflicting version.

UC-05: Operations Volume Logging & Period Locking

Use Case	Operations Volume Logging & Period Locking (Req 4.5)
Actor	Operations Staff, Operations Manager
Precondition	1. Customer contract and price list are active. 2. Operational billing period is open.
Postcondition	1. Actual operational volumes are recorded. 2. Operational records are updated with LOCKED status and cannot be modified through normal operations.
Main path (M)	1. Operations Staff enters or imports operational volume records. 2. Operations Staff reviews and reconciles the records. 3. Operations Manager reviews the operational summary and executes "Lock Period". 4. System updates the status of operational records to LOCKED. 5. Locked records become the reference for billing generation.
Alternative path	(A1) Post-Lock Edit Attempt: 1a. Operations Staff attempts to modify records whose status is LOCKED. 2a. System rejects the request unless special supervisor authorization is provided.

UC-06: Payment Slip Generation with Price Snapshot

Use Case	Payment Slip Generation with Price Snapshot (Req 4.6)
Actor	Accounting Staff, Billing Service
Precondition	1. Operational volume records for the billing period have status LOCKED. 2. Active Contract and Price List exist for the billing period scope.
Postcondition	1. Payment Slip is generated with billing line items storing both unit_price and snapshot_unit_price. 2. Direct modification after approval is prohibited and requires a formal adjustment process.
Main path (M)	1. Accountant selects Customer, Contract, and Billing Period. 2. Billing Service retrieves locked operational volume records and the corresponding active Price List. 3. Billing Service calculates billing line items and stores the applied unit price in unit_price together with snapshot_unit_price. 4. System calculates subtotal, tax, and grand total. 5. Accountant reviews the Payment Slip and submits it for approval. 6. Once approved, the Payment Slip cannot be modified directly; changes require an Adjustment Record.
Alternative path	(A1) Price List Updated After Billing: 1a. A new Price List version becomes effective after the Payment Slip has been generated. 2a. System preserves the stored unit_price and snapshot_unit_price, ensuring historical billing results remain unchanged.

UC-07: Dynamic Configurable Workflow Engine Execution

Use Case	Dynamic Configurable Workflow Engine Execution (Req 4.7)
Actor	Department Staff, Managers, Board of Directors, Workflow Engine
Precondition	1. Target document (Contract, Price List, or Billing Sheet) is in DRAFT or REVISION_REQUESTED state. 2. Workflow definition exists in the Workflow Service database.
Postcondition	1. Document advances through dynamically configured approval stages. 2. Workflow step information and approval status are maintained dynamically through the workflow_steps table without hardcoded if/else logic.
Main path (M)	1. User submits a document for approval. 2. Workflow Service creates a workflow instance and determines the current approval step. 3. Assigned approver performs an action (APPROVE, REJECT, or REQUEST_REVISION). 4. Workflow Service updates the current workflow step status and proceeds to the next step when appropriate. 5. System displays workflow execution progress based on the status of each workflow step.
Alternative path	(A1) Concurrent Approval Submission: 3a. Two approvers submit actions simultaneously for the same workflow step. 3b. System detects the conflict and rejects the duplicated action with HTTP 409 Conflict.

UC-08: Asynchronous E-Signing Integration

Use Case	Asynchronous E-Signing Integration (Req 4.8)
Actor	Board of Directors, Workflow Service, E-Sign Provider (Mock / Third-Party)
Precondition	1. Contract or Billing Sheet has completed the internal approval workflow. 2. The generated PDF document is stored in Object Storage.
Postcondition	1. Document is transmitted asynchronously for digital signing. 2. E-signing session status is maintained by the E-Sign Service until completion or failure.
Main path (M)	1. Workflow Service publishes the DOCUMENT_READY_FOR_SIGN event. 2. E-Sign Service creates an e-signing session and submits the PDF document to the external E-Sign Provider. 3. Authorized signer performs the digital signature. 4. E-Sign Provider sends the signing result through a callback. 5. E-Sign Service updates the signing status and marks the document as issued if signing succeeds.
Alternative path	(A1) E-Sign Service Failure: 4a. Callback fails or the external E-Sign Provider is temporarily unavailable. 4b. System records the failed signing status and retries the signing process without changing the document approval status.

UC-09: Asynchronous Event-Driven Notification Service

Use Case	Asynchronous Event-Driven Notification Service (Req 4.9)
Actor	System Users, Kafka Message Broker, Notification Service
Precondition	1. A domain event is generated by a business service.
Postcondition	Target users receive notifications and are able to mark them as read.
Main path (M)	1. Business Service writes a domain event to the transactional outbox. 2. Outbox Worker publishes the event to Kafka. 3. Notification Service consumes the event. 4. Notification Service creates notification records for target users. 5. Users view the notification list and mark notifications as read.
Alternative path	(A1) Notification Service Unavailable: 3a. Notification Service is temporarily unavailable. 3b. Kafka retains the messages until Notification Service resumes processing.

UC-10: Immutable System Audit Logging & Idempotency

Use Case	Immutable System Audit Logging & Idempotency (Req 4.10)
Actor	System Auditor, Audit Service, API Gateway, Redis Cache
Precondition	A state-changing API request is executed by an authenticated user.
Postcondition	1. Immutable audit information is stored. 2. Duplicate API requests are prevented using idempotency checking.
Main path (M)	1. Client sends a unique X-Idempotency-Key with each request. 2. API Gateway checks the key in Redis before forwarding the request. 3. Target microservice records audit information including User ID, Action, Resource, Previous State, Updated State, and Timestamp. 4. Audit events are published to Kafka through the Transactional Outbox pattern. 5. Audit Service stores immutable records in the audit_logs table. 6. Authorized users can search audit history by business resource.
Alternative path	(A1) Duplicate Request Submission: 2a. API Gateway detects an existing X-Idempotency-Key. 2b. Gateway rejects the duplicate request or returns the cached response.

2 Database Schema Design

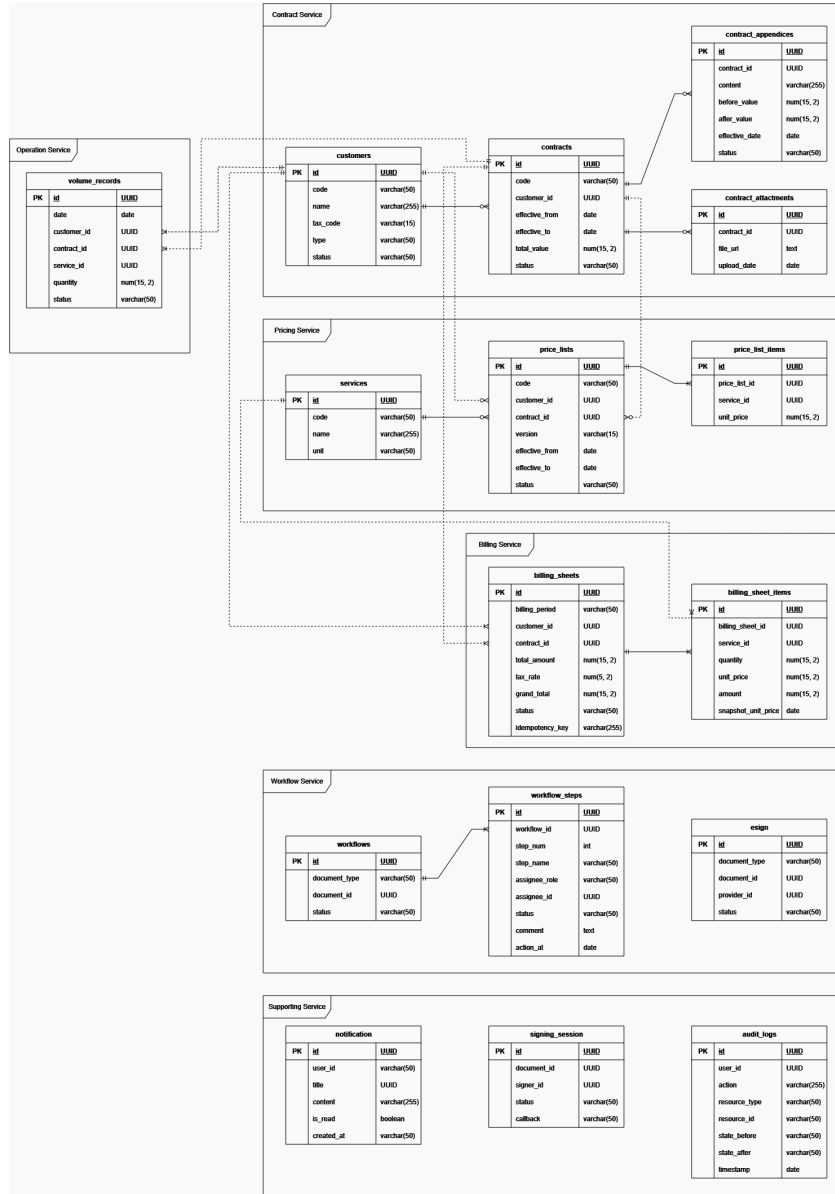


Figure 1: Database Schema

This section specifies the detailed database design for the Enterprise Business Management System under a microservices architecture.

2.1 Architectural Overview and Notation Conventions

2.1.1 Storage Architecture

The system applies the **Database-per-service** pattern. Each business Service Group owns a separate database, ensuring independent development, deployment, and operation.

2.1.2 Relationship Notation Conventions

- **Solid line:** Foreign key constraints within the service's internal database.
- **Dashed line:** Cross-database logical links between services. Data is linked and synchronized via API or Message Broker (Event-Driven).
- **Crow's Foot Notation:** Represents relationships (1–1, 1–N, ...).

2.2 Detailed Specification for Each Service Group

2.2.1 Contract Service (Customer & Contract Management Service)

Responsible for managing customer information, main contract terms, contract appendices, and attached document files.

List of tables

Table customers

- `id` (UUID – PK): Primary key, unique customer identifier.
- `code` (varchar(50)): Customer code (Unique), used for quick lookup (e.g., KH001).
- `name` (varchar(255)): Full name of the customer/enterprise.
- `tax_code` (varchar(15)): Tax code of the customer enterprise.
- `type` (varchar(50)): Customer classification (e.g., Logistics, Manufacturing, ...).
- `status` (varchar(50)): Operating status of the customer (e.g., Active, Inactive).

Table contracts

- `id` (UUID – PK): Contract primary key.
- `code` (varchar(50)): Contract code (Unique) (e.g., HD001).
- `customer_id` (UUID – FK): Customer ID representing this contract.
- `effective_from` (date): Date the contract becomes effective.
- `effective_to` (date): Date the contract expires.
- `total_value` (num(15,2)): Total value of the contract.
- `status` (varchar(50)): Contract status (e.g., Draft, Active, Expired).

Table contract_appendices

- id (UUID – PK): Primary key of contract appendix.
- contract_id (UUID – FK): Link to the main contract.
- content (varchar(255)): Brief summary of changes or additions in the appendix.
- before_value (num(15,2)): Value or unit price before adjustment.
- after_value (num(15,2)): Value or unit price after adjustment.
- effective_date (date): Date the appendix actually takes effect.
- status (varchar(50)): Approval status of the contract appendix.

Table contract_attachments

- id (UUID – PK): Attachment primary key.
- contract_id (UUID – FK): Link to the main contract owning this file.
- file_url (text): Storage path of the file on the centralized storage system.
- upload_date (date): Date the document was uploaded to the system.

Internal (physical) relationships

- customers – contracts: $1 \dots n$: Each customer can have 0 or multiple contracts.
- contracts – contract_appendices: $1 \dots n$: A contract can have multiple additional appendices.
- contracts – contract_attachments: $1 \dots n$: A contract can have multiple attached scan files.

2.2.2 Operation Service (Operations & Actual Volume Service)

Records and stores periodic actual service volumes/quantities incurred.

List of tables**Table volume_records**

- id (UUID – PK): Primary key for volume recording.
- date (date): Date the volume was recorded.
- customer_id (UUID): Customer ID incurring volume (Logical link).
- contract_id (UUID): Applied contract ID (Logical link).
- service_id (UUID): Incurred service ID (Logical link).
- quantity (num(15,2)): Actual volume/quantity executed.
- status (varchar(50)): Volume status (e.g., Draft, Confirmed, Locked).

Cross-service (logical) relationships

- `customers.id` (Contract Service) – `volume_records.customer_id`: $1 \dots n$
- `contracts.id` (Contract Service) – `volume_records.contract_id`: $1 \dots n$
- `services.id` (Pricing Service) – `volume_records.service_id`: $1 \dots n$

2.2.3 Pricing Service (Price List & Service Catalog Management Service)

Manages the catalog of services provided by the company and detailed unit prices applied to specific customers/contracts over time.

List of tables

Table `services`

- `id` (UUID – PK): Service catalog primary key.
- `code` (varchar(50)): System service code (Unique, e.g., DV001).
- `name` (varchar(255)): Service name (e.g., Warehousing, Container handling).
- `unit` (varchar(50)): Unit of measurement (e.g., Trip, Day, Container).

Table `price_lists`

- `id` (UUID – PK): Price list primary key.
- `code` (varchar(50)): Price list code (Unique).
- `customer_id` (UUID): Logical link to the customer applying this price list.
- `contract_id` (UUID): Logical link to the contract applying this price list.
- `version` (varchar(15)): Price list version (e.g., v1, v2).
- `effective_from` (date): Start date of unit price application.
- `effective_to` (date): End date of unit price application.
- `status` (varchar(50)): Status of the price list version (e.g., Draft, Effective, Expired).

Table `price_list_items`

- `id` (UUID – PK): Detailed unit price line primary key.
- `price_list_id` (UUID – FK): Physical link to the master price list.
- `service_id` (UUID – FK): Physical link to the service catalog.
- `unit_price` (num(15,2)): Unit price applied for the service in this price list.

Internal (physical) relationships

- `price_lists` – `price_list_items`: $1 \dots n$: A price list must contain at least one service unit price.
- `services` – `price_list_items`: $1 \dots n$: A service can appear in multiple detailed price lists.

Cross-service (logical) relationships

- `customers.id` (Contract Service) – `price_lists.customer_id`: $1 \dots n$
- `contracts.id` (Contract Service) – `price_lists.contract_id`: $1 \dots n$

2.2.4 Billing Service (Billing Sheet Service)

Calculates volume data and corresponding unit prices to generate billing sheets for customers according to the billing period.

List of tables

Table `billing_sheets`

- `id` (UUID – PK): Billing sheet primary key.
- `billing_period` (varchar(50)): Billing period (e.g., “08-2026”).
- `customer_id` (UUID): Customer to be billed (Logical link).
- `contract_id` (UUID): Applied contract (Logical link).
- `total_amount` (num(15,2)): Total service amount before tax.
- `tax_rate` (num(5,2)): Applied tax rate (e.g., 10.00%).
- `grand_total` (num(15,2)): Final total payable amount including tax.
- `status` (varchar(50)): Billing status (e.g., Draft, Approved, Paid).
- `idempotency_key` (varchar(255)): Anti-duplication key when submitting requests multiple times.

Table `billing_sheet_items`

- `id` (UUID – PK): Detailed billing line primary key.
- `billing_sheet_id` (UUID – FK): Physical link to the master billing sheet.
- `service_id` (UUID): Billed service type (Logical link).
- `quantity` (num(15,2)): Service consumption quantity during the period.
- `unit_price` (num(15,2)): Service unit price fixed at the time of billing.
- `amount` (num(15,2)): Calculated amount ($\text{quantity} \times \text{unit_price}$).
- `snapshot_unit_price` (date): Date of backing up this unit price snapshot from the base price system.

Internal (physical) relationships

- `billing_sheets` – `billing_sheet_items`: $1 \dots n$: Master billing sheet includes multiple detailed fee lines.

Cross-service (logical) relationships

- `customers.id` (Contract Service) – `billing_sheets.customer_id`: $1 \dots n$
- `contracts.id` (Contract Service) – `billing_sheets.contract_id`: $1 \dots n$
- `services.id` (Pricing Service) – `billing_sheet_items.service_id`: $1 \dots n$

2.2.5 Workflow Service (Approval Workflow Service)

Provides dynamic approval workflows for all document types in the system.

List of tables

Table workflows

- `id` (UUID – PK): Approval workflow primary key.
- `document_type` (varchar(50)): Type of document requiring approval (e.g., CONTRACT, BILLING_SHEET).
- `document_id` (UUID): Unique identifier of the document requiring approval (Logical link).
- `status` (varchar(50)): Overall workflow status (e.g., Running, Approved, Rejected).

Table workflow_steps

- `id` (UUID – PK): Approval step primary key.
- `workflow_id` (UUID – FK): Physical link to the overall approval workflow.
- `step_num` (int): Order of the approval step (e.g., Step 1, Step 2).
- `step_name` (varchar(50)): Approval step name (e.g., Sales Manager, Legal).
- `assignee_role` (varchar(50)): Role of the group authorized to approve this step.
- `assignee_id` (UUID): Specific user ID assigned to handle the step.
- `status` (varchar(50)): Step processing status (e.g., Pending, Approved, Rejected).
- `comment` (text): Comments, feedback, or rejection reason from the approver.
- `action_at` (date): Timestamp when the approver performed the action.

Table esign

- `id` (UUID – PK): Signature request primary key.
- `document_type` (varchar(50)): Type of document sent for signing (e.g., CONTRACT, BILLING_SHEET).
- `document_id` (UUID): Identifier of the document to be signed (Logical link).
- `provider_id` (UUID): Session connection ID of third-party e-signature service (e.g., VNeID ID, ...).
- `status` (varchar(50)): E-signature status (e.g., Waiting, Signed, Failed, Cancelled).

Internal (physical) relationships

- `workflows` – `workflow_steps`: $1 \dots n$: An approval workflow always includes sequential approval steps.

Cross-service (logical) relationships The field pair (`document_type`, `document_id`) in both `workflows` and `esign` logically links directly to business data tables of other services (such as `contracts`, `price_lists`, or `billing_sheets`) depending on the active business process.

2.2.6 Supporting Service (Support & Shared Utilities Service)

Supports supplementary system utilities including: Audit log, Transaction Notifications, and Signing Session Management.

List of tables

Table notification

- `id` (UUID – PK): Notification primary key.
- `user_id` (varchar(50)): ID of the notification recipient user, using `customers.code`.
- `title` (varchar(255)): Notification title.
- `content` (text): Detailed notification content sent to the user.
- `is_read` (boolean): Read status (True/False).
- `created_at` (varchar(50)): Notification creation timestamp.

Table signing_session

- `id` (UUID – PK): Signing session primary key.
- `document_id` (UUID): Logical link to the document needing signature.
- `signer_id` (UUID): Unique identifier of the person signing.
- `status` (varchar(50)): Status of the current signing session.
- `callback` (varchar(255)): Asynchronous response URL (Webhook) from the signing partner.

Table audit_logs

- `id` (UUID – PK): Audit record primary key.
- `user_id` (UUID): User ID who performed the action.
- `action` (varchar(255)): Description of the action (e.g., Create Contract, Approve Billing).
- `resource_type` (varchar(50)): Type of affected entity (e.g., CONTRACT, BILLING_SHEETS).
- `resource_id` (varchar(50)): Specific affected entity ID.
- `state_before` (varchar(50)): Backup of state before change (Old data).
- `state_after` (varchar(50)): Backup of state after change (New data).
- `timestamp` (date): Exact time the change occurred.

3 Database Sequence Specifications

This section presents the sequence diagrams describing the interactions among microservices, databases, and external systems for the major business workflows of the Enterprise Business Management System. These diagrams illustrate request processing, service collaboration, event-driven communication, and persistent data operations.

The following sequence diagrams correspond directly to the functional use cases presented in Chapter 4.

3.1 UC-01 Enterprise Customer Management

Figure 2 illustrates the customer registration process. After the Sales Executive submits customer information, the Customer Service validates the tax code, stores the customer profile, publishes a domain event through Kafka, and returns the successful response.

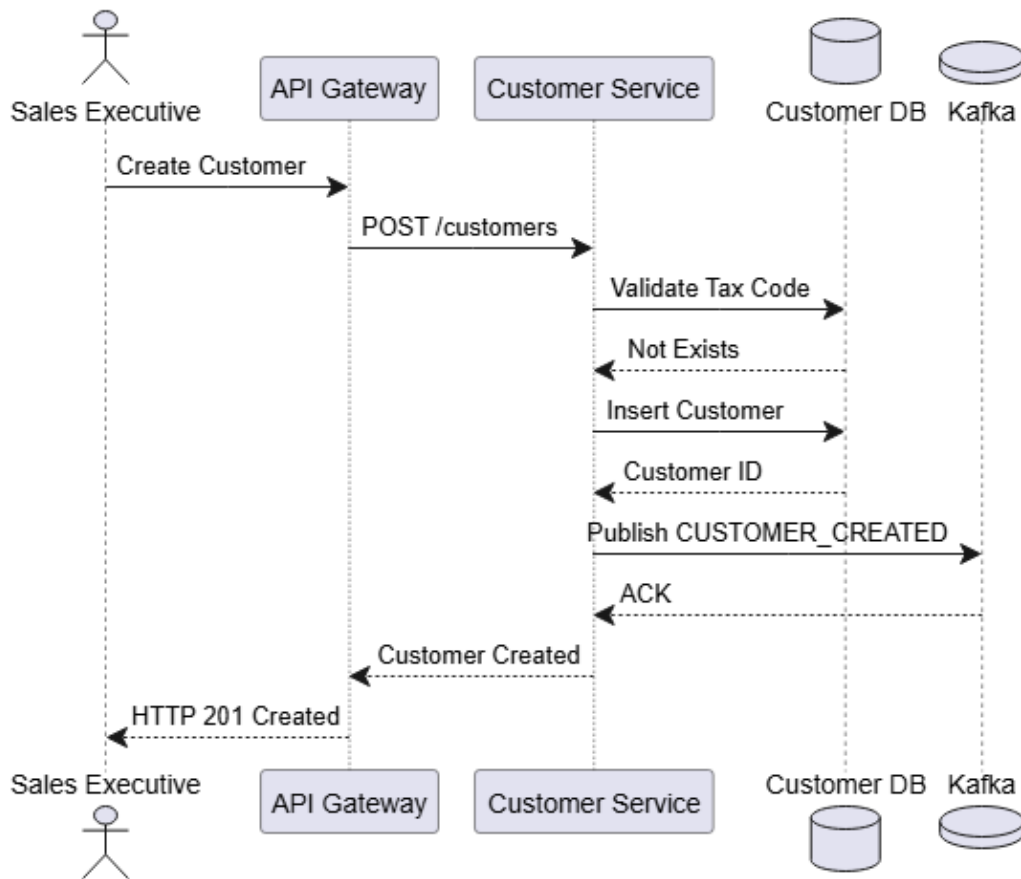


Figure 2: Sequence Diagram – Enterprise Customer Management

3.2 UC-02 Contract Lifecycle Management

Figure 3 presents the contract creation and approval workflow. The Contract Service stores the draft contract before invoking the Workflow Service to execute the configurable approval process. Once approved, the contract status is updated to ACTIVE.

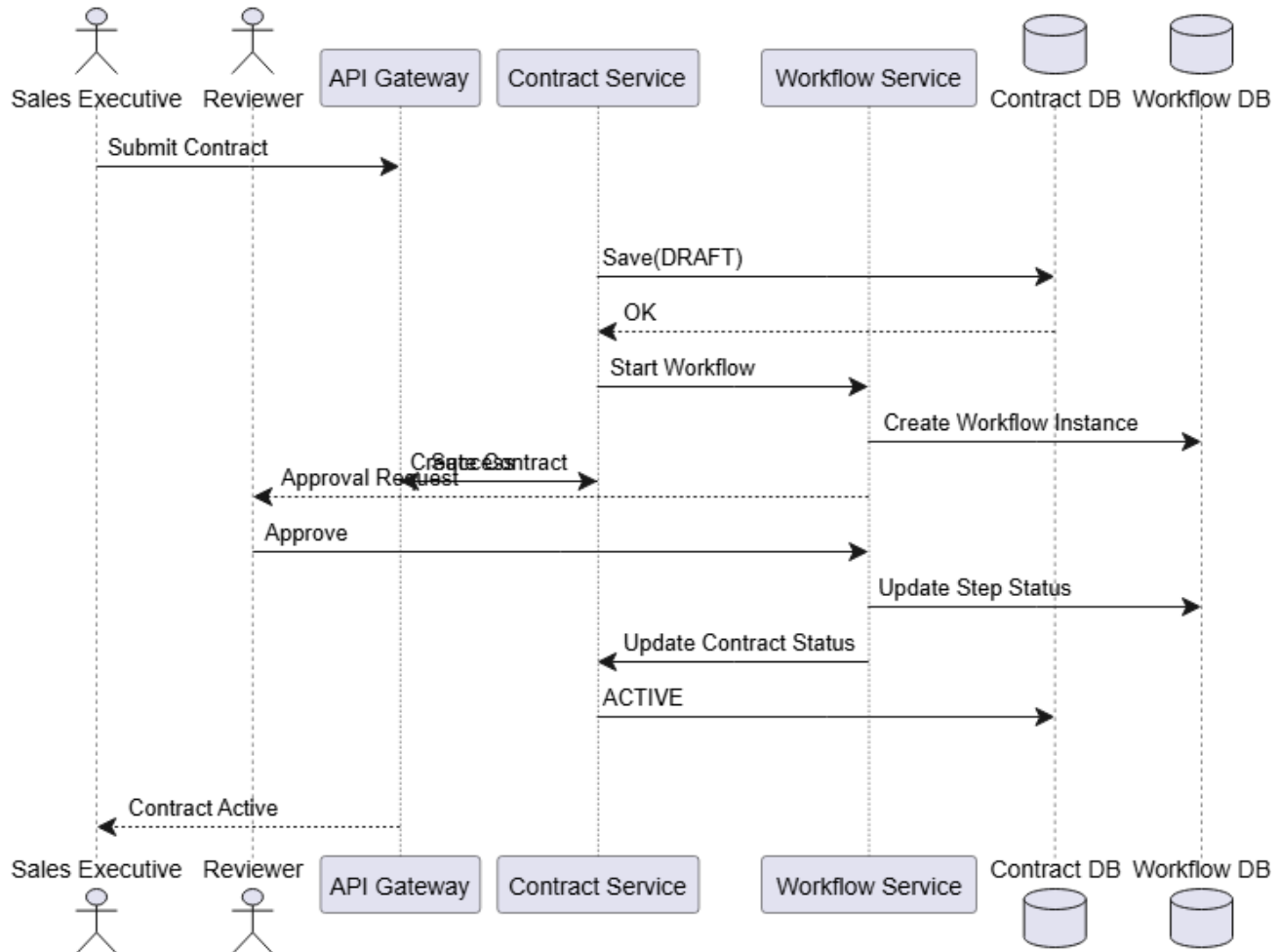


Figure 3: Sequence Diagram – Contract Lifecycle Management

3.3 UC-04 Service Price List Management

Figure 4 shows how the Pricing Service validates overlapping effective periods, creates a new price list version, supersedes previous versions, and stores the new pricing configuration.

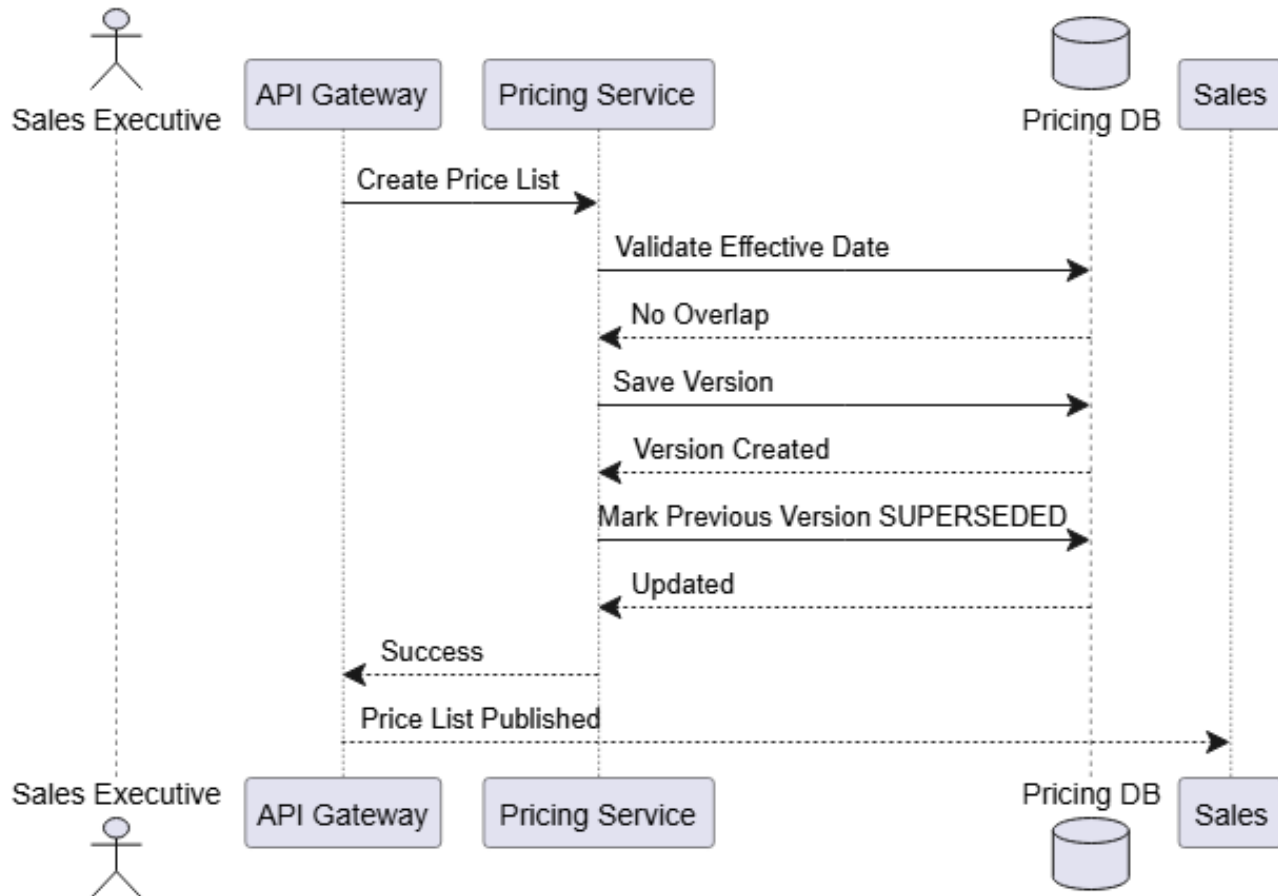


Figure 4: Sequence Diagram – Service Price List Management

3.4 UC-05 & UC-06 Billing Generation

Figure 5 describes the billing generation workflow. Billing Service retrieves locked operational volume records from Operation Service, obtains the active pricing information, calculates billing totals, stores the billing sheet, and returns the generated invoice.

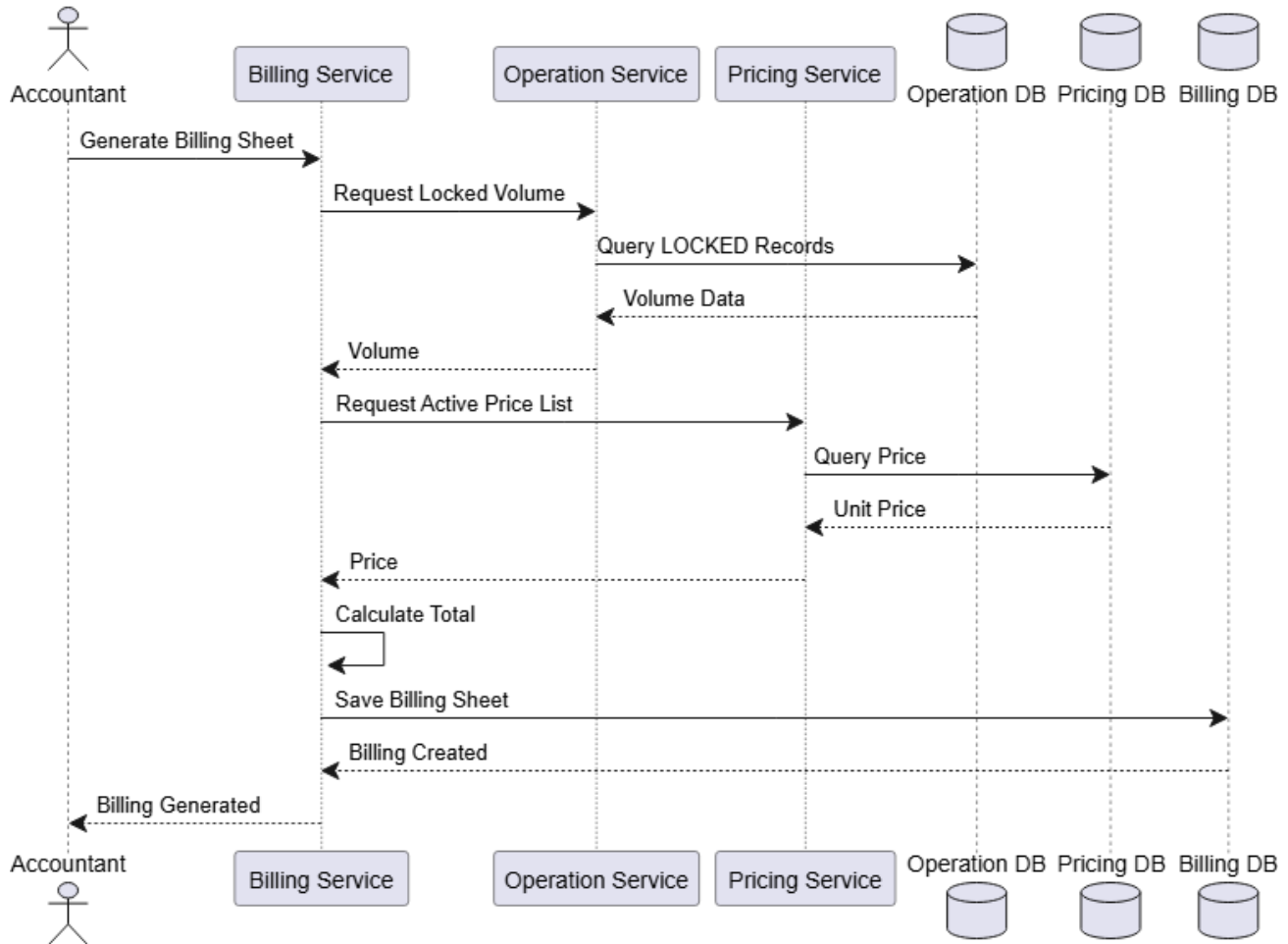


Figure 5: Sequence Diagram – Billing Generation

3.5 UC-08 Asynchronous E-Signing

Figure 6 illustrates the asynchronous digital signing process. Workflow Service publishes a signing request, the E-Sign Service creates a signing session with the external provider, receives callback notifications, and updates the signing status.

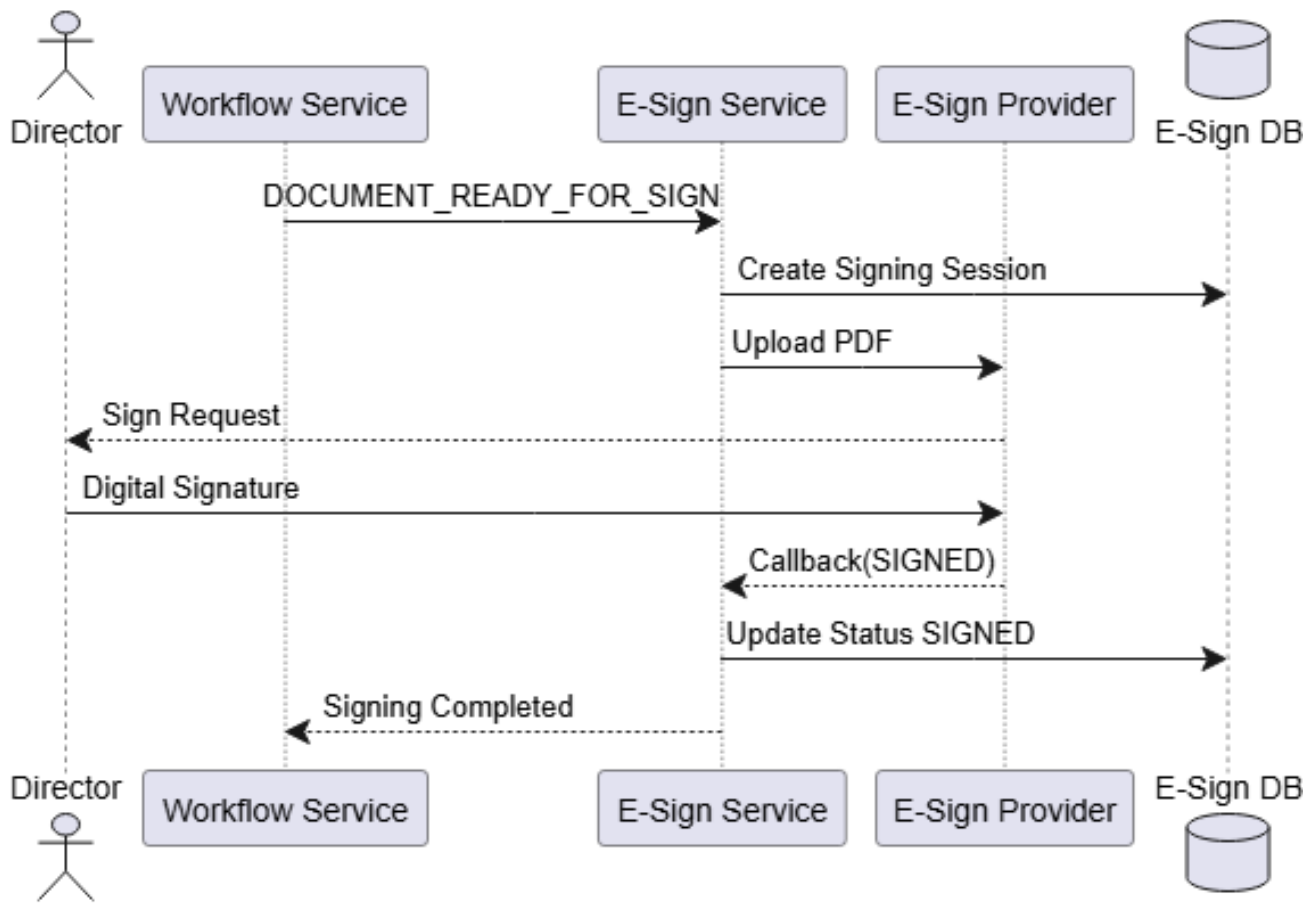


Figure 6: Sequence Diagram – Asynchronous E-Signing

3.6 UC-09 & UC-10 Notification and Audit Logging

Figure 7 presents the event-driven notification and audit workflow. API Gateway validates the idempotency key, Business Service processes the request, publishes domain events to Kafka, Audit Service records immutable logs, and Notification Service generates user notifications.

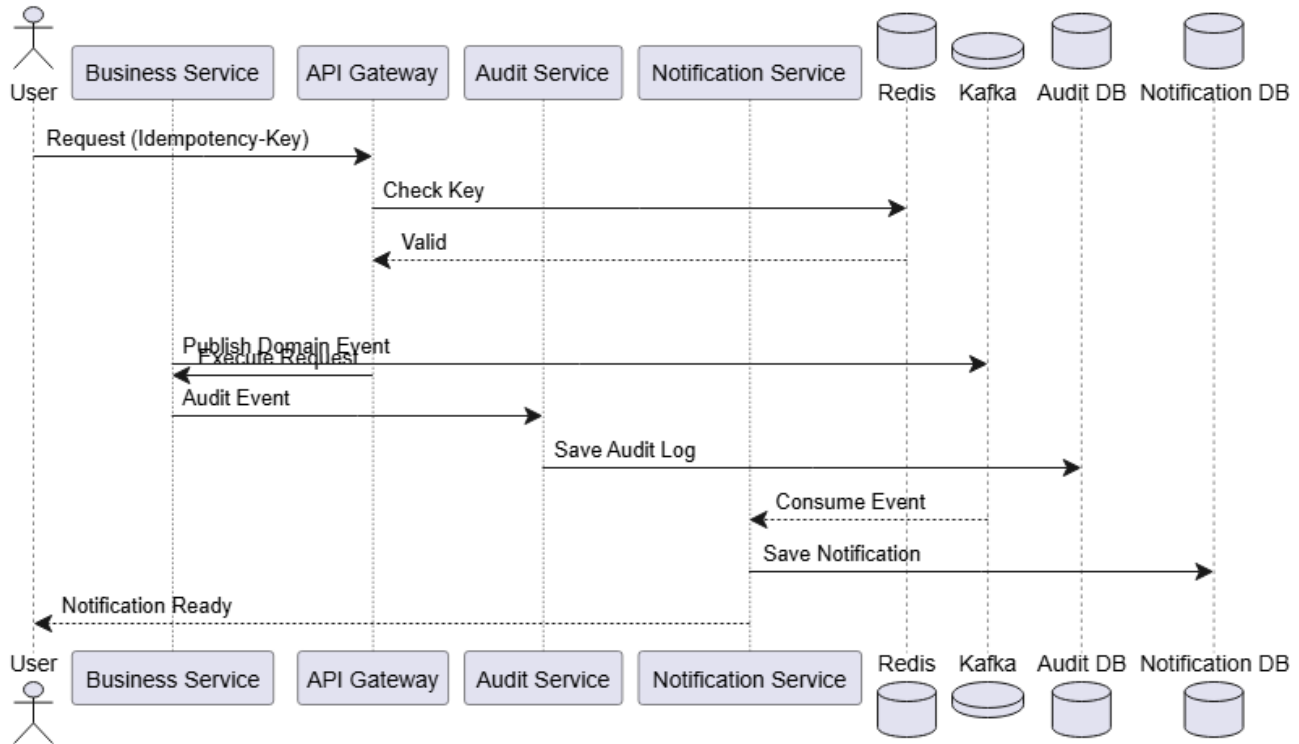


Figure 7: Sequence Diagram – Notification and Audit Logging

4 UI/UX

The user interface is designed in Figma (English PortOps style) with role-based navigation for Sales, Operations, Accounting, Legal, Board of Directors, and Admin.

Screen map (required surfaces).

- M01 Login (error state for invalid credentials)
- M02 Dashboard: waiting-for-me inbox counts; contracts/price lists nearing expiry
- M03–M04 Customers and Contracts (list/detail, FSM timeline, workflow panel, appendix create)
- M05 Price lists (version list / compare)
- M06 Volume periods (enter volumes, lock period)
- M07 Billing wizard: Draft → Calculate → Reconcile → Submit (show snapshot unit_price); e-sign / Issued banners
- M08 Approval inbox (assignee-only) with Approve / Reject / Request revision + mandatory comment
- M09 Notifications; M10 Audit log; M11 Admin users/roles and workflow templates

Figma: <https://www.figma.com/design/ejSjrtgisjfGwZZqTabAOI>